

Versionamiento Semántico

Nombre del autor

 **Tiempo estimado: 10 min**

Escuela de Ingeniería en Ciencias y Sistemas.

Segundo Semestre 2026

[Laboratorio - Nombre del Curso - Sección]

1. Marco Formativo

1.1 Valores

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Responsabilidad	Al asignar versiones claras, asumimos la responsabilidad de comunicar cambios a usuarios y equipos
Colaboración	El versionamiento facilita el trabajo en equipo mediante convenciones compartidas
Transparencia	Los changelogs y tags hacen visible el historial de cambios del proyecto

1.2 Competencia(s)

Tipo de Competencia	
Competencia General	Genera soluciones de software aplicando buenas prácticas de análisis, control de versiones y metodologías ágiles para garantizar proyectos eficientes y colaborativos en entornos virtuales de trabajo
Competencia Específica	Aplica el versionamiento semántico para gestionar cambios en el software, comunicando claramente el impacto de actualizaciones mediante el esquema Major.Minor.Patch

2. Palabras Clave

Versionamiento Semántico: Sistema de numeración (Major.Minor.Patch) que comunica el tipo e impacto de los cambios en el software.

Major: Incremento por cambios incompatibles con versiones anteriores.

Minor: Incremento por nuevas funcionalidades compatibles.

Patch: Incremento por correcciones de errores compatibles.

GitFlow: Flujo de trabajo con ramas que integra el versionamiento en releases y hotfixes.

Changelog: Documento que registra las modificaciones de cada versión.

3. Introducción

El versionamiento semántico es un sistema que permite a los desarrolladores asignar números de versión específicos a su software, facilitando la comprensión de los cambios y su impacto. Basado en un esquema de numeración estructurado, ayuda a gestionar expectativas de desarrolladores y usuarios finales, asegurando comunicación clara sobre el estado y actualizaciones del software.

4. Contenidos

Presentación de temas y subtemas.

Esquema del Versionamiento Semántico

Componentes de la versión

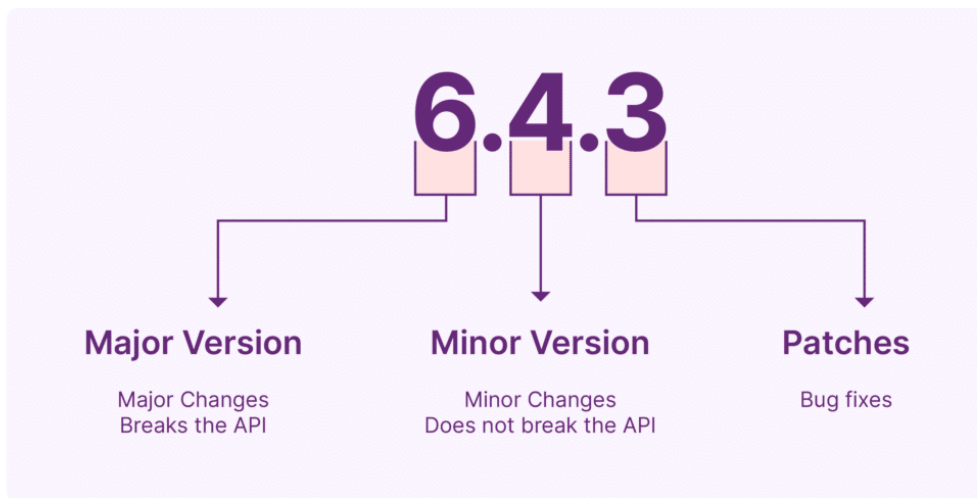
El esquema de versionamiento semántico se compone de tres números:

- **Número Mayor (Major):** Se incrementa cuando se realizan cambios incompatibles con versiones anteriores.
- **Número Menor (Minor):** Se incrementa cuando se añaden nuevas funcionalidades de manera compatible.
- **Número de Parche (Patch):** Se incrementa cuando se realizan correcciones de errores compatibles.

Ejemplo de Esquema

Un número de versión podría ser 6 . 4 . 3, donde:

- **6** es el número mayor.
- **4** es el número menor.
- **3** es el número de parche.



¿Cómo Aplicar el Versionamiento Semántico en GitFlow?

En un flujo de trabajo que utiliza Gitflow, el versionamiento semántico se aplica principalmente durante el proceso de **release**. A continuación, se presentan algunas recomendaciones:

- **Nombres de Ramas:**
 - Es esencial mantener consistencia al nombrar la rama de release. Por ejemplo, se podría usar la rama `release/1.0.0` para indicar la primera versión lista para producción.
- **Etiquetas (Tags):**
 - Al momento de realizar un lanzamiento, es fundamental etiquetar la versión en el repositorio. Utilizando el comando `git tag`, los desarrolladores pueden marcar versiones específicas, como `1.0.0`, al fusionar la rama de release en `main`.
- **Documentación del Cambio:**
 - Al fusionar cambios desde la rama de desarrollo a la rama de release, es recomendable crear un archivo de changelog que refleje las nuevas características, mejoras y correcciones de errores. Este archivo debe incluir el número de versión correspondiente.

Ejemplo de Aplicación en Gitflow

- Cuando se complete el desarrollo de una nueva funcionalidad en la rama `develop`, y esté listo para una nueva versión, los desarrolladores crearán una rama de release, por ejemplo, `release/1.1.0`.

- Al finalizar los preparativos para el lanzamiento, se fusiona la rama de release en main y se crea la etiqueta (tag) correspondiente 1.1.0.
- Si se requiere un hotfix después del lanzamiento, se crearía una rama (por ej. hotfix/functionality_name-fix), se realizaría la corrección y luego se fusionaría en main y develop, asegurando que el número de versión se actualice de acuerdo con los cambios realizados.

Caso de estudio

Librería de JavaScript

Planteamiento: Una librería de JavaScript, llamada "WidgetLib", fue desarrollada para facilitar la creación de widgets en aplicaciones web. Originalmente, la versión fue lanzada como 1.0.0.

Desglose del Problema:

- Con el tiempo, los desarrolladores implementaron nuevas funcionalidades, como la integración con APIs de terceros y mejoras en la documentación.
- Sin embargo, los desarrolladores también identificaron que algunas actualizaciones anteriores rompían la compatibilidad con aplicaciones que usaban versiones anteriores de la librería.

Solución:

- Los desarrolladores decidieron adoptar el versionamiento semántico para clarificar la naturaleza de las actualizaciones.
- Las versiones se comenzaron a etiquetar adecuadamente:
 - La adición de nuevas funcionalidades llevó a la versión 1.1.0.
 - Posteriormente, se realizaron correcciones de errores, resultando en 1.1.1.
 - Finalmente, se decidió reestructurar partes del código, lo que llevó a un cambio mayor, incrementando la versión a 2.0.0.
- Esta implementación ayudó a los usuarios de "WidgetLib" a entender mejor qué esperar de cada versión y facilitó la adaptación a cambios más significativos.

5. Conclusiones

- El versionamiento semántico proporciona un marco claro y estructurado para comunicar los cambios en un software. Al seguir esta convención, los equipos de desarrollo pueden informar a los usuarios y otros desarrolladores sobre la naturaleza de los cambios realizados, ya sean nuevas características, mejoras o correcciones de errores.
- Utilizar un sistema de versionamiento semántico ayuda a manejar las dependencias en proyectos de software.

6. Referencias

1. Preston-Werner, T. (2013). *Semantic Versioning 2.0.0*. <https://semver.org>
2. Chacon, S. & Straub, B. (2014). *Pro Git*. Apress. <https://git-scm.com/book>